

LLM → Prolog → LLM: Multi-Step Verified Generation Through Alternating Neural-Symbolic Computation

Eliminating Hallucination by Construction via Provenanced Integer Knowledge Bases, Triveritas Evaluation, and Adaptive Goal Decomposition

Registry: [[CKS-MATH-138-2026](#)]

Series Path: [[CKS-0-2026](#)] → [[CKS-MATH-0-2026](#)] → [[CKS-MATH-1-2026](#)] → [[CKS-MATH-10-2026](#)] → [[CKS-MATH-104-2026](#)] → [[CKS-MATH-137-2026](#)] → [[CKS-MATH-138-2026](#)]

Parent Framework: [[CKS-0-2026](#)]

DOI: 10.5281/zenodo.18960010

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [[CKS-NEXT-1-2026](#)]

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [[CKS-LEX-12-2026](#)]

ABSTRACT

Current large language models generate output through unconstrained token prediction — a process with no verification step, no logical consistency checking, no provenance tracking, and no structured knowledge representation. The result is “hallucination”: outputs that are statistically plausible but factually wrong, logically inconsistent, or untraceable to any source. We present an alternative architecture in which an integer-trained

LLM ([CKS-MATH-134-2026]) alternates with a Prolog-based verification engine at every step of generation. The LLM handles what neural networks do well: fuzzy input comprehension and creative pattern selection. Prolog handles what logical systems do well: consistency verification, goal decomposition, constraint enforcement, and provenance tracking. We prove: (1) **Hallucination is eliminated by construction** — every generated fact traces to provenanced sources in the knowledge base; outputs without provenance are structurally impossible, (2) **Term-based tokenization replaces BPE** — tokens are typed, structured Terms carrying their grammatical role, not arbitrary byte-pair fragments, (3) **Three-dimensional evaluation** — every claim is evaluated on logical validity (L), mathematical coherence (M), and empirical anchoring (E) via the Triveritas criterion, (4) **Materiality gating** — the Scales Method prevents computation on non-material concerns, (5) **Adaptive sequencing** — the Pseudo-Socratic Method determines the number and focus of generation steps based on continuous state assessment, (6) **The knowledge base replaces the context window** — a persistent, provenanced, version-filtered fact store that never forgets and never degrades, (7) **Domain eating** — new knowledge domains are added by writing parsers and rules, not by retraining the neural network. From first principles through complete architecture. The LLM is the interface. The knowledge base is the mind.

Central claim: The hallucination problem is not a deficiency of neural networks. It is the inevitable consequence of generating output without verification. Interleaving neural creativity with logical verification at every step produces output that is verified by construction, not evaluated after the fact.

I. THE PROBLEM: UNCONSTRAINED GENERATION

1.1 How Current LLMs Generate Output

A current large language model receives a token sequence and predicts the next token. The prediction is a probability distribution over the vocabulary, computed by:

1. Embedding the input tokens as float vectors.
2. Passing through transformer layers (attention + feedforward).
3. Projecting to a vocabulary-sized vector of logits.
4. Applying softmax to produce probabilities.
5. Sampling or selecting the highest-probability token.
6. Appending to the sequence and repeating.

At no point in this pipeline does the system:

- Verify that the generated token is factually correct.
- Check that the growing output is internally consistent.
- Confirm that the output traces to any source.

- Evaluate whether the output satisfies the user’s actual goal.
- Decompose a complex request into verified substeps.

The output is unconstrained token prediction. The model produces whatever is statistically most likely given the input. If the training data contains contradictory information, the model may reproduce either side. If the training data is sparse in some area, the model fills the gap with statistically plausible but factually wrong completions. This is called hallucination.

1.2 Why Hallucination Cannot Be Fixed Post-Hoc

The industry has attempted to fix hallucination through:

RLHF (Reinforcement Learning from Human Feedback): Trains the model to produce outputs humans prefer. Does not verify correctness — only preference. A confidently stated falsehood that sounds authoritative may score higher than a hedged truth.

Retrieval-Augmented Generation (RAG): Retrieves text chunks from a database and stuffs them into the context window. The retrieval is approximate (float vector similarity). The chunks may contradict each other. The model may ignore the retrieved text. No verification occurs.

Chain-of-thought prompting: Asks the model to show its reasoning. The reasoning is also generated by unconstrained token prediction. The model can produce convincing but invalid reasoning chains.

Tool use: The model calls external tools (calculators, databases, APIs). The model decides when to call tools and how to interpret results — both decisions are unconstrained token prediction. The model can misuse tools, misinterpret results, or fail to call them when needed.

Each approach adds a mechanism on top of unconstrained generation. None changes the fundamental architecture. The generation step remains unverified.

1.3 The Alternative: Verification at Every Step

If generation alternates with verification at every step, hallucination becomes structurally impossible. Not reduced. Not mitigated. Impossible.

The generated output at each step is a set of structured facts. The verification engine checks each fact against the knowledge base. Facts that are consistent with known, provenanced information pass. Facts that contradict the knowledge base are rejected. Facts with no provenance are flagged as unverified.

The output consists only of facts that passed verification. An unverified fact cannot appear in the output because the emission step only processes verified facts. This is not a filter applied to generated text — it is a structural property of the architecture.

II. TERM-BASED TOKENIZATION

2.1 The Problem with BPE

Byte-pair encoding (BPE) is the standard tokenization method for current LLMs. It operates on raw bytes, merging frequent pairs into tokens. The process is:

1. Start with individual bytes (256 tokens).
2. Count all adjacent byte pairs in the corpus.
3. Merge the most frequent pair into a new token.
4. Repeat until the target vocabulary size is reached.

BPE is format-blind. It does not know that `pub` is a keyword, that `fn` is a keyword, that `i32` is a type, or that `allocator` is an identifier. It knows only that certain byte sequences co-occur frequently. The result:

- `allocator` might be split into `alloc` + `ator` — two fragments with no individual meaning.
- `i32` inside a comment and `i32` as a type annotation are the same token despite having completely different roles.
- The model must learn from scratch that keywords, types, identifiers, and operators are different categories.

This is wasted capacity. The source language already has this structure. Zig has a tokenizer. C has a tokenizer. English has parts of speech. The structure exists in the source material and BPE discards it.

2.2 Terms as Tokens

In this architecture, the token is a typed Term:

```
TermType = enum(i32) {  
  // Structure  
  container, // opens a concept: function, struct, sentence  
  delimiter_open, // ( [ ' '  
  delimiter_close, // ) ] ' '  
  separator, // , ; . : |  
  // Code  
  name, // identifier: max, bruce, allocator  
  kind, // category: fn, struct, noun, verb  
  type_ref, // type: i32, bool, []u8  
  keyword, // control: if, while, return, try
```

```

operator, // arithmetic/logic: + - * == > <
// Values
number, // integer literal
text_literal, // string content
// Logic
predicate, // Prolog predicate name
rule, // rule reference
fact, // fact assertion
variable, // unification variable
// Reference
reference, // entity/index pointer
// Spatial (from MetaProlog)
vector2,
rectangle,
entity,
transform,
}

```

Each token carries its grammatical role in its `type` field. The model does not need to learn that `i32` is a type — the `TermType` says `.type_ref`. The model does not need to learn that `fn` is a keyword — the `TermType` says `.kind`. The structure is in the token.

2.3 Tokenization as Parsing

Tokenization is no longer statistical byte-pair merging. It is parsing by domain-specific parsers that produce structured Terms:

Zig source is parsed by Zig’s own tokenizer:

```

pub fn max(a: i32, b: i32) i32 {
→ { .keyword, "pub" }
{ .kind, "fn" }
{ .name, "max" }
{ .delimiter_open, "(" }
{ .name, "a" }
{ .separator, ":" }

```

```
{.type_ref, "i32"}  
{.separator, ","}  
{.name, "b"}  
{.separator, ":"}  
{.type_ref, "i32"}  
{.delimiter_close, ")"}  
{.type_ref, "i32"}  
{.delimiter_open, "{"}
```

English text is parsed by a word-level tokenizer with part-of-speech tagging:

The serpent hissed a warning.

→ {.kind, "article", "the"}

{.name, "noun", "serpent"}

{.kind, "verb", "hissed"}

{.kind, "article", "a"}

{.name, "noun", "warning"}

{.separator, "."}

Every Term carries provenance:

```
{.type_ref, "i32", source=99002, offset=2104, version=15}
```

The source file, byte offset, and version are attached at parse time. They travel with the Term through the entire system. They are never discarded.

2.4 What This Changes

The model's embedding layer maps a TermType (small enum, approximately 20 categories) plus a value (index within that type's vocabulary) to an integer vector. The model starts with grammatical structure baked into the representation and learns relationships between structured Terms — not relationships between arbitrary byte fragments.

Capacity that BPE-trained models spend learning tokenization, part-of-speech categories, and syntactic roles is freed for learning the actual task: what Terms follow what Terms in what contexts.

III. THE PROVENANCED KNOWLEDGE BASE

3.1 Structure

The knowledge base is a set of Prolog facts, each carrying multi-dimensional metadata:

```
Fact {  
  predicate: Text, // what this fact asserts  
  args: []Term, // the structured arguments  
  source: Source, // who/what produced this fact  
  timestamp: i32, // when (turn number or absolute)  
  confidence: Confidence, // stated, inferred, derived, speculative  
  verification: Verification, // high, medium, low, none  
  provenance: Provenance, // source_id, byte offset, length  
  supersedes: ?FactId, // what fact this replaces (if correction)  
  version: u32, // source material version  
}
```

Every fact traces to its origin. A fact extracted from the Zig standard library carries the file hash, byte offset, and Zig version. A fact stated by the user carries the source `.user_prompt` and the turn number. A fact derived by Prolog carries the source `.prolog_derivation` and the IDs of the facts it was derived from.

3.2 Multi-Dimensional Indexing

Following the multi-dimensional information indexing framework ([?]), facts are not stored as flat claims. They carry their full context:

Temporal specificity: When was this true? A fact about Zig 0.14 API is true for version 14. It may be superseded by a different fact for version 15. Both coexist in the KB with their version tags. No contradiction — temporal resolution.

Source tracking: Who said this? The user (high confidence for their own intent), the training corpus (confidence depends on source quality), or Prolog derivation (confidence depends on the facts it derived from).

Verification depth: Can this be independently checked? A fact from the Zig stdlib source is verifiable by reading the source file at the stored offset. A fact from the user's prompt is authoritative for their intent. An inference by the LLM is speculative until Prolog validates it.

Supersession: When the user corrects a fact, the old fact is not deleted. It is superseded. Both exist in the KB. Queries return the current (unsuperseded) version. The history is preserved for audit.

3.3 Version Filtering

Every fact carries the version of its source material. Queries are filtered by version:

```
valid_fact(Fact) :-  
    provenance(Fact, Source),  
    source_version(Source, Version),  
    query_version(RequestedVersion),  
    Version == RequestedVersion.
```

A query for Zig 0.15 API usage never sees Zig 0.14 facts. The old facts exist in the KB — they are not deleted — but they are excluded from the search space by the version filter. This eliminates the version confusion problem where LLMs trained on mixed-version data produce stale API calls.

3.4 The Knowledge Base Replaces the Context Window

Current LLMs maintain context as a fixed-size token buffer. When it fills, old tokens are discarded. The model’s attention mechanism can only reach tokens within the buffer. Information beyond the buffer is permanently inaccessible for the duration of the session. Between sessions, everything is lost.

The knowledge base has none of these limitations:

No size limit. Facts accumulate. RAM-resident facts are accessed in nanoseconds. Overflow facts are stored on disk and loaded on demand. An LRU policy manages which facts are hot.

No positional degradation. A fact from the first turn is equally accessible as a fact from the current turn. There is no attention decay. Prolog matches on predicates and arguments, not on position in a sequence.

No session boundary. The KB persists. Sessions are views into the KB, not separate conversations. Multiple sessions can read and write the same KB simultaneously.

No information loss from compaction. If RAM pressure requires eviction, specific facts are evicted based on LRU access patterns. The eviction is surgical — remove individual facts, not “forget the first half of the conversation.” Evicted facts remain on disk and can be reloaded.

IV. THE ALTERNATING PIPELINE

4.1 Overview

Generation proceeds through N steps, where each step consists of:

1. **LLM phase:** The neural network processes the current context and produces structured Terms.
2. **Prolog phase:** The verification engine evaluates the Terms against the knowledge base.
3. **Update phase:** Verified facts enter the KB. Rejected facts are discarded with an explanation. The context for the next step is updated.

N is not a fixed constant. It is determined by goal decomposition (Section V). Simple requests may complete in 3 steps. Complex requests may require 15 or more. The Pseudo-Socratic sequencer (Section VI) determines when to advance, when to retry, and when to terminate.

4.2 Step 1: Prompt Comprehension

The user provides a natural language prompt. The LLM’s first task is to convert this fuzzy input into structured Terms.

User: “make me a function that takes two numbrs and retruns the biggr one”

LLM produces Terms:

```
{.predicate, "create"}
{.container, "function"}
{.number, 2} → param count
{.type_ref, "number"} → param type
{.predicate, "returns"}
{.predicate, "larger"} → behavior
```

The LLM performs two operations here. First, fuzzy matching: “functon” → “function”, “numbrs” → “numbers”, “biggr” → “bigger”. This is pattern matching against known vocabulary — a strength of neural networks. Second, classification: each meaningful word is assigned a TermType. This is a small classification problem (approximately 20 categories), not open-ended text generation.

The LLM is not generating language. It is labeling incoming words with types and mapping them to known atoms.

4.3 Step 2: Prolog Verification of Understanding

Prolog receives the Terms and checks them against the knowledge base:

% Does “function” as a container match known patterns?

valid_container(function) :- known_construct(function, source=?S). ✓

% Does “number” as a type reference resolve?

```

valid_type(number) :-
type_alias(number, i32, source=?S). ✓ (or ambiguous — might be f32)
% Is “larger” a known comparison behavior?
valid_behavior(larger) :-
known_comparison(larger, source=?S). ✓

```

If verification passes, the understood intent is recorded as facts in the KB:

```

intent(create, timestamp=1, source=user_prompt, confidence=stated).
target(function, timestamp=1, source=user_prompt, confidence=stated).
param_count(2, timestamp=1, source=user_prompt, confidence=stated).
param_type(number, timestamp=1, source=llm_inference, confidence=inferred).
return_behavior(larger_of_params, timestamp=1, source=llm_inference, confidence=inferred).

```

Note the confidence levels. The user stated they want a function — that is `confidence=stated`. The LLM inferred the param type is “number” — that is `confidence=inferred`. If ambiguity exists (number could be i32, f32, or generic), Prolog flags it for clarification before proceeding.

4.4 Step 3: Goal Decomposition

Prolog decomposes the verified intent into a plan:

```

goal(create_function).
step(1, define_signature, requires=[name, params, return_type]).
step(2, resolve_types, requires=[concrete_types_for_all_params]).
step(3, select_body_pattern, requires=[behavior, matching_algorithm]).
step(4, emit_code, requires=[all_previous_steps_verified]).

```

Each step has explicit requirements. The plan is itself a set of facts in the KB, inspectable and modifiable.

4.5 Steps 4-N: Alternating Generation and Verification

For each step in the plan:

LLM generates: Given the current KB state and the current step’s requirements, the LLM selects from available options. For “select body pattern matching the behavior `larger_of_params`”, the LLM considers patterns it has seen in training:

Option A: if (a > b) return a; return b;

Option B: return if (a > b) a else b;

Option C: `return [?](a, b);`

The LLM selects Option C because `@max` is the most common pattern in the Zig training data for this behavior.

Prolog verifies: Does `@max` exist in the Zig standard library at the requested version? Does it accept two arguments of the parameter type? Does it return the expected type?

`builtin(max, params=[anytype, anytype], returns=anytype, source=zig_015_builtins, offset=...). ✓`

Verification passes. The selection is recorded in the KB:

`body_pattern(max_builtin, step=3, source=llm_selection, confidence=inferred, verified_by=prolog, verification_source=zig_015_builtins).`

If verification fails: The rejection reason becomes context for a retry.

Prolog: “[?] requires explicit error handling in Zig 0.15. Missing ‘try’ or ‘catch’.”

LLM receives: “Step 3 rejected. Reason: missing error handling for [?]”.

`Retry with error handling."`

LLM generates new option incorporating the constraint.

No backward pass through previous steps. The current step retries with enriched context. Forward only.

4.6 Final Step: Deterministic Emission

After all steps are verified, Prolog assembles the complete set of verified Terms:

`emit({.keyword, "pub"}).`

`emit({.kind, "fn"}).`

`emit({.name, "max"}).`

`emit({.delimiter_open, "("}).`

`emit({.name, "a"}).`

`emit({.separator, ":"}).`

`emit({.type_ref, "i32"}).`

`emit({.separator, ","}).`

`emit({.name, "b"}).`

`emit({.separator, ":"}).`

`emit({.type_ref, "i32"}).`

`emit({.delimiter_close, "}")).`

```

emit({.type_ref, "i32"}).
emit({.delimiter_open, "{"}).
emit({.keyword, "return"}).
emit({.name, "[?]}").
emit({.delimiter_open, "("}).
emit({.name, "a"}).
emit({.separator, ","}).
emit({.name, "b"}).
emit({.delimiter_close, ")"}).
emit({.separator, ";"}).
emit({.delimiter_close, "}"})

```

The emission is deterministic. Each verified Term has exactly one text representation. A simple lookup converts Terms to text with appropriate whitespace:

```

zig
pub fn max(a: i32, b: i32) i32 {
  return [?](a, b);
}

```

The LLM does not write this code. The emitter writes it from verified Terms. Hallucination is structurally impossible in the emission step because only verified Terms are emitted.

V. TRIVERITAS EVALUATION

5.1 Three Dimensions of Validity

Every claim in the knowledge base — whether from the user, the LLM, or Prolog derivation — is evaluated on three independent dimensions following the Triveritas criterion ([?]):

L — Logical validity. Is the claim internally consistent? Does it contradict other claims in the KB? Is the derivation chain valid?

M — Mathematical coherence. Are the quantities possible? Do the numbers work? Are the types compatible? Can the claimed operation produce the claimed result?

E — Empirical anchoring. Does the claim trace to a verifiable source? How many independent sources support it? What is the verification level of those sources?

5.2 Prolog Implementation

```
dimension(Claim, logical, pass) :-  
    not(contradicts(Claim, ?AnyOtherClaim)),  
    valid_derivation_chain(Claim).  
dimension(Claim, logical, fail(Reason)) :-  
    contradicts(Claim, Other),  
    Reason = contradiction(Claim, Other).  
dimension(Claim, mathematical, pass) :-  
    quantities_in(Claim, Quantities),  
    all_computable(Quantities),  
    types_compatible(Claim).  
dimension(Claim, mathematical, untested) :-  
    no_quantities_in(Claim).  
dimension(Claim, empirical, pass) :-  
    provenance(Claim, Sources),  
    length(Sources, N),  
    N > 0,  
    max_verification(Sources, Level),  
    Level >= medium.  
dimension(Claim, empirical, fail) :-  
    provenance(Claim, Sources),  
    length(Sources, 0).
```

5.3 Classification

```
knowledge(Claim) :-  
    dimension(Claim, logical, pass),  
    dimension(Claim, mathematical, pass),  
    dimension(Claim, empirical, pass).  
strong_opinion(Claim) :-  
    two_of_three_pass(Claim).  
rationalist_trap(Claim) :-
```

```

dimension(Claim, logical, pass),
dimension(Claim, mathematical, pass),
dimension(Claim, empirical, fail).
empiricist_trap(Claim) :-
dimension(Claim, empirical, pass),
dimension(Claim, logical, fail).
scholastic_trap(Claim) :-
dimension(Claim, logical, pass),
dimension(Claim, mathematical, fail),
dimension(Claim, empirical, fail).

```

The system does not merely report “correct” or “incorrect.” It reports which dimensions pass and fail, what specific evidence supports or undermines each dimension, and what historical pattern the failure matches. The failure patterns are named after the epistemological traditions that characteristically exhibit them ([?]).

VI. SCALES METHOD: MATERIALITY GATING

6.1 The Materiality Threshold

Before the system spends computation evaluating a claim or generating a response, the Scales Method ([?]) applies a materiality gate:

```

material(Claim) :-
changes_outcome(Claim),
scope(Claim, Percentage),
Percentage > 5.

```

A claim is material if it changes the outcome of the current goal and affects a non-negligible percentage of cases. Non-material claims are tagged but not deeply evaluated. This prevents the system from spending cycles on irrelevant concerns.

6.2 Scope Quantification

Material claims are quantified by scope:

```

scope(Claim, Percentage) :-
total_cases(Claim, Total),
affected_cases(Claim, Affected),

```

Percentage = (Affected * 100) / Total.

severity(Claim, negligible) :- scope(Claim, P), P < 5.

severity(Claim, minor) :- scope(Claim, P), P >= 5, P < 20.

severity(Claim, significant) :- scope(Claim, P), P >= 20, P < 50.

severity(Claim, major) :- scope(Claim, P), P >= 50, P < 80.

severity(Claim, critical) :- scope(Claim, P), P >= 80.

These are integer percentages. Exact. Comparable. Not float probabilities with unknown precision.

6.3 Fruit of the Plant Validation

The Scales Method includes outcome-based validation:

fruit_of_plant(Hypothesis, strong) :-

stated_goals(Hypothesis, Goals),

observed_outcomes(Hypothesis, Outcomes),

consistent(Goals, Outcomes),

intermediate_steps_consistent(Hypothesis),

no_simpler_explanation(Hypothesis).

fruit_of_plant(Hypothesis, fails) :-

stated_goals(Hypothesis, Goals),

observed_outcomes(Hypothesis, Outcomes),

not(consistent(Goals, Outcomes)).

If the generated output achieves the stated goal (e.g., the code compiles and produces correct output), the generation is validated by its fruit. If the output fails to achieve the goal, the failure is traced through the verification chain to identify which step's verification was insufficient.

VII. PSEUDO-SOCRATIC SEQUENCING

7.1 State Assessment

The Pseudo-Socratic Method ([?]) provides the execution controller that determines how many LLM $\leftrightarrow$ Prolog steps to take and what each step should focus on.

Before each step, the system assesses the current state:

comprehension(topic, solid) $\rightarrow$ advance to next step

comprehension(topic, gap) → backfill prerequisite
comprehension(topic, confused) → clarify before proceeding
comprehension(topic, not_introduced) → introduce when prerequisites solid

7.2 Convergent Mode

When the goal is known (e.g., “write a function”), the system operates in convergent mode:

```
mode(convergent).  
goal(create_function, max, [i32, i32], i32).  
next_action(advance, step_N) :-  
    all_previous_steps_verified,  
    current_step_requirements_met.  
next_action(retry, step_N) :-  
    current_step_rejected(Reason),  
    retry_count(step_N, Count),  
    Count < max_retries.  
next_action(backfill, prerequisite) :-  
    current_step_requires(Concept),  
    comprehension(Concept, gap).  
next_action(complete) :-  
    all_steps_verified,  
    fruit_of_plant(Goal, strong).
```

7.3 Divergent Mode

When the goal is open-ended (e.g., “explore approaches to this design problem”), the system operates in divergent mode:

```
mode(divergent).  
exploring(design_problem).  
available_paths(Paths) :-  
    findall(Path, viable_option(Path), Paths).  
next_action(explore, Path) :-  
    available_paths(Paths),  
    score_by_utility(Paths, Scored),
```


best(Scored, Path).
next_action(terminate) :-
sufficient_exploration,
satisfactory_outcome_found.

7.4 The N Steps Are Not Arbitrary

The number of LLM $\leftrightarrow$ Prolog alternations is determined by the goal structure, not by a hyperparameter. A simple request (“what type does [?] return?”) may require one LLM step (parse query) and one Prolog step (look up fact). A complex request (“design an allocator with these constraints”) may require many steps, each decomposed and verified.

The Pseudo-Socratic sequencer monitors progress and adapts:

- If a step succeeds, advance.
 - If a step fails, retry with the failure reason in context.
 - If retries exhaust, escalate (ask user for clarification or simplify the goal).
 - If all steps succeed, validate fruit of the plant and emit.
-

VIII. DOMAIN EATING

8.1 Adding Knowledge Without Retraining

In current LLM architectures, adding a new knowledge domain requires retraining or fine-tuning the neural network on data from that domain. This costs compute time proportional to model size and dataset size — days to weeks on GPU clusters.

In this architecture, adding a domain is:

1. Obtain source material for the domain.
2. Write a domain-specific parser that produces Terms with provenance.
3. Write Prolog rules encoding the domain’s patterns and constraints.
4. Run the parser on the source material, producing provenanced facts.
5. Validate consistency of the ingested facts.
6. Load facts and rules into the KB.

The neural network is not retrained. The KB grows. New facts are immediately queryable. New rules are immediately applicable.

8.2 Domain-Specific Parsers

Each domain has a parser that produces the universal Term format:

Zig source → Terms:

Uses Zig's own tokenizer. Emits Terms with TermTypes `.keyword`, `.name`, `.type_ref`, etc. Provenance is the source file hash and byte offset.

C headers → Terms:

Uses a C tokenizer. Emits function signatures, type definitions, macro definitions as facts. Provenance is the header file hash and byte offset.

English text → Terms:

Uses word-level tokenization with part-of-speech classification. Emits Terms with TermTypes `.kind` (noun, verb, adjective), `.name`, `.separator`. Provenance is the book/document hash and byte offset.

Slang dictionaries → Terms:

Parses structured entries (word, definition, votes, date). Emits Terms with confidence derived from vote ratios and temporal metadata.

Any new domain → Terms:

Write a parser. The output format is always the same: typed Terms with provenance. The rest of the system does not change.

8.3 Domain Rules

Each domain has a set of Prolog rules encoding valid patterns:

`% Zig: a function call is valid if the function exists and types match`

`valid_call(Fn, Args) :-`

`function(Fn, source=?S),`

`source_version(?S, RequestedVersion),`

`match_params(Fn, Args).`

`% English: adjectives precede nouns`

`valid_modifier(Terms) :-`

`find_by_pos(Terms, adjective, Adj),`

`find_by_pos(Terms, noun, Noun),`

`immediately_precedes(Adj, Noun, Terms).`

`% Japanese: SOV with particles`

`valid_sentence(Terms) :-`

```

subject(Terms, S),
particle(Terms, [I], after(S)),
object(Terms, O),
particle(Terms, [I], after(O)),
verb(Terms, V, final).

```

Rules are written by domain experts (or generated by an LLM and reviewed). They are small — typically a few hundred rules per domain. They are explicit, inspectable, and correctable. If a rule is wrong, it is identified by the provenance of the incorrect output and fixed directly.

8.4 Cross-Domain Queries

Because all domains produce the same Term format and all facts live in the same KB, cross-domain queries work automatically through shared predicates:

```

% A Zig function wraps a C function
wraps(ZigFn, CFn) :-
  calls_via_cimport(ZigFn, CFn, source=zig_source),
  function(CFn, source=c_header).

% Same plant in two languages
same_entity(BabylonianName, ModernName) :-
  entity_name(Plant, BabylonianName, source=babylonian_corpus),
  entity_name(Plant, ModernName, source=modern_botany).

```

No special cross-domain mechanism is needed. Prolog unification over shared predicates connects facts across domains naturally.

IX. WHAT THE LLM DOES AND DOES NOT DO

9.1 The LLM Does

Fuzzy input comprehension. Mapping misspelled, ambiguous, colloquial human input to structured Terms. This is pattern matching — the LLM’s core strength.

Creative option selection. When Prolog presents multiple valid options for a generation step, the LLM selects the most appropriate based on patterns in training data. “What’s the most idiomatic Zig pattern for this?” is a question the LLM can answer well.

Anomaly recognition. Noticing that a combination of facts, while individually valid, is unusual or interesting. “This set of constraints is similar to problem X, which was solved by approach Y.” This cross-pattern recognition is a neural network strength.

9.2 The LLM Does Not

Write output text. The emitter writes output from verified Terms. The LLM never produces the final text.

Enforce constraints. Prolog enforces constraints. The LLM proposes; Prolog disposes.

Track state. The KB tracks state. The LLM does not maintain a mental model of the conversation — the KB does, as explicit facts.

Verify its own output. The LLM’s output is always verified by Prolog before it affects anything. Self-verification is a known failure mode of neural networks. External verification by a logical system eliminates it.

Remember across turns. The KB remembers. The LLM processes the current step’s context, which includes relevant KB facts marshaled by Prolog. The LLM does not need long-range memory because the KB provides it.

9.3 Implications for Model Size

Because the LLM handles only fuzzy comprehension and creative selection — not knowledge storage, not constraint enforcement, not state tracking, not output generation — it can be substantially smaller than a general-purpose LLM that must handle all of these tasks in its weights.

A 100M-1B parameter model may suffice for the LLM component because:

- Knowledge is in the KB, not the weights.
- Grammar and syntax are in the Prolog rules, not learned from data.
- Provenance is in the fact metadata, not encoded implicitly.
- Consistency is enforced by Prolog, not hoped for from statistics.

The model needs only enough capacity to map fuzzy input to structured Terms and to select between options based on training patterns. This is a dramatically smaller task than what current LLMs attempt.

X. THERE ARE NO HALLUCINATIONS

10.1 Redefining the Failure Mode

In this architecture, the failure mode historically called “hallucination” does not exist. What exists instead is **insufficient path coverage**.

When the system produces a wrong answer, the cause is traceable:

1. The query walked the Prolog fact graph.
2. It followed the strongest available path.

3. The strongest path led to an incorrect conclusion.
4. The reason: insufficient training data in that specific area to build a better path.

Every fact on the path has provenance. The source files, byte offsets, versions, and confidence levels are all inspectable. The wrong answer has a specific, identifiable cause: these specific sources, at these specific offsets, contributed these specific facts, which formed this specific path.

10.2 The Fix Is Always the Same

Identify the gap in the fact graph. Add better sourced training data for that specific area. Reparse with the domain parser. The new facts enter the KB. The path strengthens. The default hit becomes a correct hit.

Not RLHF. Not prompt engineering. Not temperature tuning. Not hoping the next training run fixes it. Identify the gap, fill it with provenanced data, done.

10.3 The System Tells You What It Does Not Know

Path coverage is queryable:

`path_coverage(Query, Coverage) :-`

`findall(Path, valid_path(Query, Path), Paths),`

`length(Paths, Coverage).`

Coverage 0: no training data. The system reports “I have no information about this” rather than generating a plausible-sounding fabrication.

Coverage 1: one path. Fragile. The system reports the answer with its single source and confidence level.

Coverage 3+: multiple independent paths. Convergent. High confidence.

The system does not guess. It indexes. It reports what the index contains with full provenance. Where the index is empty, it says so.

XI. PERSISTENT SESSION ARCHITECTURE

11.1 Sessions as Views

A session is not a conversation that starts and ends. A session is a view into the KB with an active context filter:

Session {

id: u64,

active_context: []Text, // topics currently in focus

```

created: timestamp,
last_active: timestamp,
}

```

Multiple sessions can run simultaneously against the same KB. Each sees the full KB but filters by its active context. Facts created in one session are visible to all sessions through shared predicates.

11.2 No Degradation Over Time

A conversation in the current architecture degrades as the context window fills. In this architecture, the session does not degrade because:

- The KB is not a fixed-size buffer. It grows.
- Facts do not fall off the end. They persist.
- Contradictions are resolved by supersession, not by context contamination.
- The Pseudo-Socratic state tracker maintains explicit state, not inferred state.
- Prolog detects inconsistencies structurally, not through attention patterns.

Turn 10,000 is as coherent as turn 1. The KB is larger and richer, but its consistency properties are maintained by the same Prolog rules that operated at turn 1.

11.3 LRU Memory Management

```
fact_metadata(FactId, last_accessed, access_count).
```

```
eviction_score(FactId, Score) :-
```

```
fact_metadata(FactId, LastAccessed, AccessCount),
```

```
age(LastAccessed, Age),
```

```
Score = Age * 1000 / (AccessCount + 1).
```

```
evict_if_needed :-
```

```
ram_pressure_high,
```

```
eviction_score(FactId, Score),
```

```
Score > threshold,
```

```
move_to_disk(FactId).
```

Hot facts stay in RAM. Cold facts move to disk. Nothing is deleted. The system manages memory pressure through principled eviction, not through forgetting.

XII. FALSIFICATION CRITERIA

This paper is falsified if any of the following are demonstrated:

F1. The alternating LLM $\leftrightarrow$ Prolog pipeline produces hallucinated output — output that does not trace to any provenanced fact in the KB and was not flagged as unverified. This would demonstrate a structural flaw in the verification architecture.

F2. The Term-based tokenization produces worse language modeling performance than BPE tokenization at the same model size and training data volume. This would demonstrate that the structural information in Terms is not useful or is actively harmful to learning.

F3. The Prolog verification step is too slow to be practical — more than $10 \times$ slower than unconstrained generation for the same output length. This would demonstrate that the verification overhead makes the architecture impractical.

F4. The domain eating process (parser + rules + validation) takes longer than retraining the LLM on the same domain data. This would undermine the claimed efficiency advantage of structured knowledge addition.

F5. The persistent KB degrades in quality over extended use — producing less accurate or less consistent outputs at turn 10,000 than at turn 100. This would demonstrate that the consistency properties claimed for the KB do not hold in practice.

F6. The materiality gating (Scales Method) filters out claims that later prove critical, at a rate higher than the miss rate of ungated systems on the same inputs. This would demonstrate that the efficiency gain costs accuracy.

Each criterion specifies a concrete test. The paper stands until a specific test demonstrates a specific failure.

XIII. FUTURE WORK

13.1 Implementation

The complete pipeline described in this paper requires integration of components that currently exist independently: the VFR integer LLM ([CKS-MATH-134-2026], working toy), the MetaProlog engine (shipping in production in Silo), and the evaluation frameworks (Triveritas, Scales, Pseudo-Socratic — documented and field-tested).

The integration work is engineering, not research. The interfaces are specified: Terms in, Terms out, all integers, same address space.

13.2 Logic Sparring

The architecture supports a “logic sparring partner” application where the Pseudo-Socratic sequencer drives a multi-turn evaluation of the user’s reasoning. Claims are

evaluated on Triveritas dimensions. Weaknesses are identified structurally. The system challenges on specific dimensions rather than generating generic responses.

13.3 Multi-Domain Knowledge Integration

The domain eating architecture enables continuous expansion of the KB. Domains of interest include programming languages, natural languages, formal logic, mathematics, domain-specific knowledge bases (medicine, law, engineering), and historical corpora with temporal metadata.

13.4 Collaborative Knowledge Building

The user's input enriches the KB. Corrections improve it. The system's knowledge grows through use. Multiple users with different expertise can contribute to the same KB, each adding provenanced facts from their domain. The KB becomes a shared, verified, growing knowledge base rather than a static training artifact.

XIV. CONCLUSION

The hallucination problem is solved by not having it.

Current LLMs generate output through unconstrained token prediction and then attempt to detect and mitigate errors after the fact. This approach has failed because the generation step has no verification, no provenance, no logical structure, and no consistency checking.

The architecture presented here eliminates the generation step as traditionally understood. There is no unconstrained token prediction that produces the final output. Instead:

- The LLM converts fuzzy input to structured Terms. (Neural strength: pattern matching.)
- Prolog verifies the Terms against a provenanced knowledge base. (Logical strength: consistency.)
- The LLM selects from verified options. (Neural strength: creative choice.)
- Prolog verifies the selection. (Logical strength: constraint enforcement.)
- The Pseudo-Socratic sequencer determines when to advance. (Adaptive strength: state awareness.)
- The Scales Method determines what is worth evaluating. (Efficiency strength: materiality gating.)
- The Triveritas criterion determines the quality of each claim. (Evaluation strength: three-dimensional validity.)
- Deterministic emission produces output from verified Terms. (Structural strength: no unverified output possible.)

Every fact in the system traces to a provenanced source. Every derivation is inspectable. Every verification is repeatable. Every step is exact integer arithmetic on exact integer facts.

The LLM is the interface. The knowledge base is the mind. The Prolog engine is the conscience. The emission is the voice.

They do not hallucinate. They index. They verify. They emit.

The math compiles. The code runs. The architecture is specified.

Axioms first. Axioms always.

Q.E.D.

Verification: Working components: VFR LLM (toy), MetaProlog (production), evaluation frameworks (documented)

Tokenization: Term-based, replacing BPE, carrying TermType and provenance

Knowledge Base: Persistent, provenanced, version-filtered, multi-dimensional indexed

Evaluation: Triveritas (L/M/E), Scales (materiality/scope/fruit), Pseudo-Socratic (state/sequencing)

Hallucination: Eliminated by construction — unverified facts cannot be emitted

Domain eating: Parser + rules + validation, no neural retraining required

Sessions: Views into persistent KB, no degradation, no forgetting

Pipeline: LLM $\rightarrow$ Prolog $\rightarrow$ LLM, N steps determined by goal decomposition

The LLM is the interface.

The knowledge base is the mind.

The Prolog engine is the conscience.

They do not hallucinate. They index.

[[CKS-MATH-138-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-138-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-MATH-137-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-137-2026>

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[[CKS-LEX-12-2026](#)] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/papers/LEX-12-2026>

[[CKS-MATH-134-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-134-2026>